

varp python API documentation

Instance functions

```
varp.new(options)
```

Create a new varp instance from a list of options

- (varp.**size**, size)
Initial variable table size
- (varp.**qtype**, varp.lifo|varp.fifo|varp.recursive)
use lifo/fifo strategy in bcp
- (varp.**xref**, x)
use cross references if x is True
- (varp.**hash**, x)
use hash table for clauses if x is True
- (varp.**edge**, x)
use edge tables for 2-clauses if x is True

```
varp.clone(vp, options)
```

Clone the varp instance using setting options from new with the follow additions.

- (varp.**level**, l)
clone bindings up until level 'l'
- (varp.**set**, varp.**delta**)
clone clauseset DELTA
- (varp.**set**, varp.**gamma**)
clone clauseset GAMMA
- (varp.**set**, varp.**beta**)
clone clauseset BETA
- (varp.**set**, varp.**alpha**)
clone clauseset ALPHA
- (varp.**queue**, x)
clone bcp queue if x is True

```
varp.info(vp, item)
```

Get varp information

- varp.**bcp_counter**
- varp.**conflict_counter**
- varp.**max_conflicting**
- varp.**num_conflicting**
- varp.**number_of_variables**
- varp.**number_of_clauses**
- varp.**number_of_edges**
- varp.**number_of_dead_clauses**
- varp.**number_of_dead_edges**
- varp.**number_of_learnt_clauses**

- varp.**number_of_bound_variables**
- varp.**number_of_subst_variables**
- varp.**number_of_unbound_variables**
- varp.**clause_n_counter**
- varp.**clause_2_counter**
- varp.**clause_3_counter**
- varp.**clause_d_counter**
- varp.**edge_2_counter**
- varp.**edge_d_counter**
- varp.**size**
- varp.**qtype**
- varp.**max_level**
- varp.**min_level**
- varp.**max_bound**
- varp.**literal_size**
- varp.**literal_integer**
- varp.**value_packing**
- varp.**edge**
- varp.**xref**
- varp.**hash**
- varp.**phase**
- varp.**use_phase**

```
varp.config(vp, item, value)
```

Set configurable items in varp

- varp.**max_conflicting**
set max number of conflicts during bcp ($\leq$ MAX_CONFLICTING=1024)
- varp.**xref**
turn on (True) or off (False) cross reference handling
- varp.**hash**
turn on (True) or off (False) hash table handling
- varp.**qtype**
set style of literal queueing in bcp
where value is one of
varp.**lifo** | varp.**fifo** | varp.**recursive**

```
varp.add_variable(vp [,is_atom])
```

Create a new variable. The variable is returned as an index to the next available variable in the variable table. Mark the new variable as atom if `is_atom` is True. The atom status may later be queried with `variable info`. `is_atom` defaults to True.

```
varp.value(vp, x)
```

Return variable value as varp constant

varp.**t** | varp.**f** | varp.**undefined**

```
varp.bind(vp, x, [,leve])
```

Bind variable x to True. If level is given then that level is used to the variable is bound on that level else the variable is bound on the current level as set with `varp.set_level`.

```
varp.decide(vp, x [,level])
```

Bind variable x to True and mark x as a decision variable. If level is given then that level is used to the variable is bound on that level else the variable is bound on the current level as set with `varp.set_level`.

```
varp.subst(vp, x, y)
```

```
varp.implication_clause(vp, x)
```

Return the clause index for the clause where x became a unit.

```
varp.implication_level(vp, x)
```

Return the bind level for x, where it decided/bound or unit.

```
varp.implication_pos(vp, x)
```

Return the position where x is found in the implication clause.

```
varp.conflicting_clause(vp, i)
```

Return the i'th conflicting clause during the last bcp. The number of conflicting clauses that can be returned is **num_conflicting**.

```
varp.is_variable(vp, x)
```

Return True if literal x is unbound. Return False otherwise

```
varp.is_bound(vp, x)
```

Return True if literal x is bound, through substitution, to an other variable. Return False otherwise.

```
varp.is_equal(vp, x, y)
```

Check if literal x and literal y are the equal, that is bound to the same variable or are bound to the same constant.

```
varp.set_level(vp, level)
```

Set current level to 'level'. Note that level 0 is treated as constant level. So be careful when setting level to 0.

```
varp.keep_level(vp, l)
```

Keep all bindings on level l by removing the undo information.

```
varp.move_level(vp, src, dst)
```

Move bindings from level 'src' to level 'dst'. Normally src level will be hight that dst level but it possible to move (with warning) to a high level as wll.

```
varp.undo_level(vp, l)
```

Undo all bindings on level 'l'

```
varp.undo(varp)
```

Undo bindings typically after an nbind. Undo will undo all bindings until a decision and flip the variable if not already flipped.

```
varp.bcp(varp [, [x1,...,xn] [,all]])
```

Run value propagation. Return True if no contradiction is found, False otherwise.

[EXPERIMENTAL]

If literals x1..Xn are given they are checked for "turbo" rule, that is if all clauses that xi is a part of are true regardless of the value of xi. If 'all' is true then all xi's must be true for the rule to hold. If turbo rule is successful then varp.**turbo** is returned.

[EXPERIMENTAL]

```
varp.nbcv(varp)
```

decide and bind next unbound variables until either no more variables to bind, return True, or a contradiction is reached, then return False. nbcv can be use with undo to implement a tight loop for simple backtracking.

```
def bt(v):  
    while !varp.nbcv(v):  
        if varp.undo(v) == False:  
            return False; # contradiction  
        return True; # model
```

```
varp.add_clause(vp, [x1,...,xn] [,clause_set])
```

Create a new clause, given as a literal list and return the new clause index. All variables indices must already have been created by calling add_variable. The clause create is installed in one of four clause sets: varp.**delta**, varp.**gamma**, varp.**alpha**, varp.**beta**.

The varp.**delta** clause-set is use to store the "problem" formula clauses while varp.**gamma** is used for storing learnt clauses. However the conflict clauses created by varp.conflict are create in varp.**alpha** and then, by user, moved into varp.gamma.

```
varp.get_clause(vp, cix, x | varp.undefined, raw)
```

Retrive a clause as list given the clause index 'cix'.
If literal x is given then literal x is removed from the clause list returned. if raw is True then literals bound on level=0 are also return as normal, otherwise they are remove if False or the clause is dead and empty list is returned (fixme).

```
varp.find_clause(vp, [x1,...,xn])
```

Check if the clause [x1,...,xn] exist among the clause sets.
return Clause index if found, return False otherwise.

```
varp.compress_clause(vp, cix | [x1,...,xn])
```

Return a compressed version of the clause [x1,...,xn],
or the clause given by the clause index cix.
It writes a utf8 like code with 0x80 bit for continuation bit and
7-bits per byte for integer value. The LSB is coded as the literal sign.

-1000 is translated to unsigned by

```
2*1000 + 1 = 0xb11111010001
```

which is divided in groups of 7 bits starting from LSB

```
(1)1010001, (0)0001111
```

while 1000 is translated to

```
2*1000 = 0xb11111010000
```

which is divided in groups of 7 bits starting from LSB

```
(1)1010000, (0)0001111
```

Then sequence of coded literals are then terminated with a zero.

```
varp.clause_info(vp, cix, item)
```

Get information about clause given by clause index cix

- varp.**length**
- varp.**jump**
- varp.**status**
- varp.**watch0**
- varp.**watch1**
- varp.**watch**

```
varp.variable_info(vp, x, item)
```

Get information about variable x

- varp.**implication**
- varp.**implication_clause**

- varp.**implication_pos**
- varp.**level**
- varp.**phase**
- varp.**is_atom**
- varp.**degree**
- varp.**symbol**

```
varp.literal_info(vp, x, item)
```

Get information about literal x

- varp.**degree**
- varp.**user**
- varp.**edge**
- varp.**symbol**

```
varp.del_clause(vp, cix | [x1,...,xn])
```

Delete clause cix or [x1,...,xn] from clause sets.

```
varp.clean_clause(vp, cix)
```

Cleanup clause by removing all false literals on level 0.
if clause is contradictory then exception is raised, else
varp.**ok** is returned.

```
varp.clean_edges(vp, x)
```

Remove x edges, that is clauses on form [-x,y] where y
is constant.

```
varp.get_clauses(vp, cix, skip, raw)
```

Return a list of literals given by clause index cix.
Remove the literal skip from the returned list also
remove literals on level 0 if raw is False.

```
varp.get_decision(vp, level)
```

Get literal on decision level.

```
varp.get_undo_state(vp, level)
```

Return undo state

- varp.**set**
- varp.**toggle**
- varp.**done**
- varp.**undef**

```
varp.get_bindings(vp, level, clauseinfo, tail, tuple)
```

```
varp.get_nbindings(vp, count clauseinfo, trail)
```

```
varp.get_number_of_bindings(vp, level)
```

```
varp.order_sort(vp, key1, key2, arg)
```

```
varp.order_first(vp, [x1,...,xn])
```

```
varp.order_last(vp, [x1,...,xn])
```

```
varp.next_unbound(varp [, last])
```

```
varp.queue_first(varp)
```

```
varp.queue_next(vp, x)
```

```
varp.queue_clear(varp)
```

```
varp.add_symbol(vp, x, string|term)
```

```
varp.find_symbol(vp, string|term)
```

```
varp.use_clause(vp, cix)
```

```
varp.bump(vp, x, n)
```

```
varp.subscribe(vp, flags)
```

Flags

- varp.**variable**
- varp.**atom**
- varp.**number_of_variables**
- varp.**number_of_bound_variables**
- varp.**number_of_subst_variables**
- varp.**number_of_clauses**
- varp.**number_of_dead_clauses**
- varp.**max_level**
- varp.**max_bound**
- varp.**min_level**

```
varp.clauseset_size(vp, set)
```

where set is one of varp.**delta**, varp.**gamma**,
varp.**alpha**, varp.**beta**

```
varp.clauseset_offset(vp, set)
```

get offset where set is one of varp.**delta**, varp.**gamma**,
varp.**alpha**, varp.**beta**

```
varp.clauseset_offset(vp, set, offset)
```

set offset where set is one of varp.**delta**, varp.**gamma**,
varp.**alpha**, varp.**beta**

```
varp.clauseset_sort(vp, set)
```

get offset where set is one of varp.**delta**, varp.**gamma**,
varp.**alpha**, varp.**beta**

```
varp.clauseset_first(vp, set)
```

```
varp.clauseset_next(vp, set)
```

```
varp.set_user_count(vp, x, count)
```

Set user value for literal x to count.

```
varp.conflict(vp, level, bump, i)
```

Do conflict analysis, called with level where the conflict number i was found
and the bump factor that is applied to variables involved in the conflict.
Return value is a clause index in clause-set varp.**alpha**. This
clause may then be minimized and later moved to varp.**gamma**.

```
varp.minimize(vp, cix)
```

Minimize clause, may be called after varp.conflict and requires
that literals and levels are set like after the conflict.

```
varp.move_clause(vp, cix, set)
```

Move clause cix to clause-set 'set'.
This function is currently limited to clause in varp.**alpha**
and set must be varp.**GAMMA**

```
varp.mark_literals(vp, [x1,...,xn] | (x1,...,xn))
```

Mark variables x1..Xn with MARK0

```
varp.mark_intersect(vp, [x1,...,xn] | (x1,...,xn))
```

Add MARK1 to all variables x1..xn marked with MARK0.
Then variables marked with both MARK0 and MARK1 are
kept while variables kept with only MARK0 are removed

```
varp.mark_intersect_var(vp, x, tuple, [x1,...,xn] | (x1,...,xn)4)
```

```
varp.get_marked(vp, x)
```